

Hardware & Embedded Systems

Jonathan Jacob Koshy

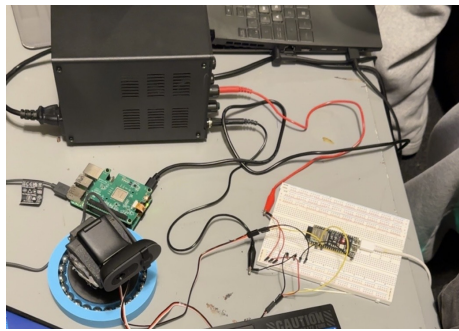
Electrical Engineering, University of Waterloo

johnkoper12@gmail.com [Portfolio](#) [GitHub](#)

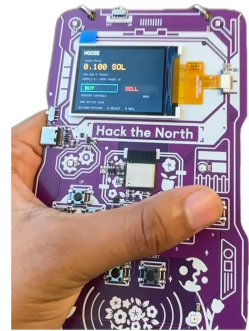
I build embedded systems from the board up: sensor interfaces, firmware, and the software that makes the hardware useful. These projects cover PC VR, wearable sensing, vehicle and flight-control firmware, and devices with tight memory and timing constraints.



jjkVR headset



remembR bench setup



Badge Market

Inside the builds

Project	Engineering focus	Page
jjkVR	STM32 PCB, sensor fusion, USB HID, SteamVR	2
AeroCardia	Sensor drivers, RTOS queues, BLE OTA, calibration	3
Midnight Sun	Vehicle safety FSM, CAN motor control, FOTA	4
UWARG	Flight-control mixing, PID tuning, LTE modem driver	5
remembR	Edge inference, detection queues, persistent object memory	6
Badge Market	ESP-NOW, state synchronization, DMA rendering	7
ESP32 media controller	LVGL, TCP framing, RTOS queues, RGB565 artwork	8

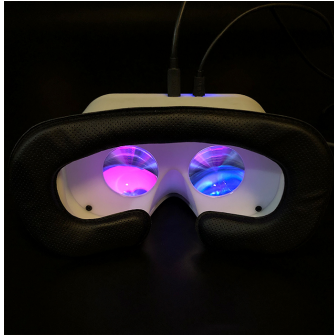
01 / PERSONAL PROJECT

jjkVR

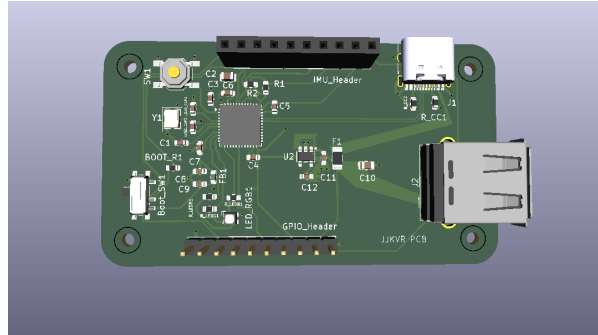
STM32F411 / C / ICM-20948 / TF-Luna / KiCad / USB HID

GitHub: [JJKSweaty/jjkVR](#) Driver references: [Relativity](#), [HadesVR](#)

I built a custom 2K, 120 Hz VR headset with a two-layer STM32F411 tracking PCB, IMU sensor fusion, and SteamVR integration. The PC drives the display through its video interface; the microcontroller handles sensor acquisition and USB pose reports.



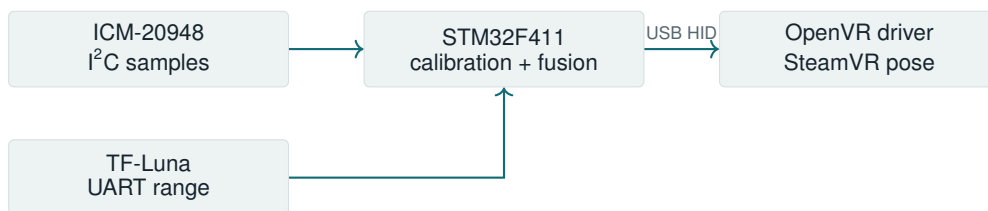
Headset prototype with the display illuminated.



Custom tracking PCB with power regulation, sensor headers, and test points.

Board design and pose tracking

The board integrates USB-C, fused 5 V display power, 3.3 V regulation, and an expansion header for another tracking reference. Firmware samples and scales ICM-20948 accelerometer, gyroscope, and magnetometer data over I²C. FastIMU quaternion fusion supplies orientation for USB HID reports to a SteamVR driver adapted from Relativity.



A defined firmware-to-PC interface

The firmware schedules IMU reads every 5 ms and HID reports every 10 ms. Startup calibration estimates stationary sensor offsets. A fixed 64-byte report carries the quaternion, optional position, validity flags, and sensor status. Coordinates are mapped before transmission so the PC driver does not apply a second axis conversion.

4DoF tracking and its limits

Three rotational axes come from the IMU; TF-Luna ranging adds one experimental forward translation axis relative to a stationary baseline. Lateral and vertical position stay fixed. Checksummed UART frames keep range acquisition separate from the IMU bus. Range correction is gated on heading alignment because turning toward another surface can change distance without head translation.

Current stage

The headset provides 4DoF tracking for seated experiments. Working towards a V2 with full 6DoF with an external positional reference

02 / AEROCARDIA INTERNSHIP

AeroCardia: AeroStim Wearable

ESP32-S3 / C++ / FreeRTOS / BLE / Altium

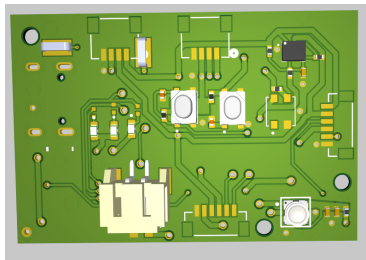
I developed sensor firmware and hardware for a cardiovascular and respiratory monitoring prototype. My work covered IMU, PPG, and temperature drivers, a queued acquisition-to-BLE pipeline, secure firmware updates, and an oxygen-sensor PCB with persistent calibration.



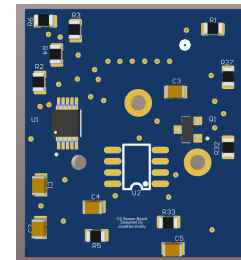
Wearable enclosure prototype, side view.



Front airflow interface and sensor housing.



ESP32-S3 main board.



Oxygen-sensor PCB with calibration EEPROM.

50 Hz acquisition with a separate BLE path

I wrote C++ drivers for IMU, PPG, and temperature sensors, enabling 50 Hz biosignal capture. FreeRTOS queues decouple sampling from radio transmission, allowing acquisition to continue through temporary BLE delays. Optimizing the sampling and buffering paths reduced dropped samples by 25% during continuous monitoring.

The 20 ms sampling period makes queue capacity and drain rate part of the timing budget. Temporary delays can be buffered; sustained radio congestion still requires an explicit sample-drop policy.

Calibration stored with the sensor

I designed the oxygen-sensor PCB in Altium with onboard EEPROM for calibration data that persists across power cycles. Keeping coefficients with the sensor assembly supports integration without depending on volatile settings. Compact placement and supply decoupling helped fit the board into the wearable.

Secure BLE updates with recovery

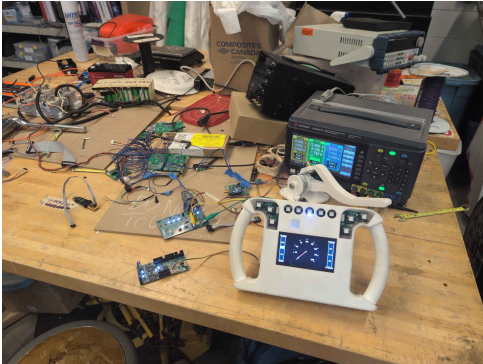
I implemented secure BLE OTA updates with image verification and rollback, bringing update time below one minute. Wireless updates avoid opening the enclosure, while rollback provides a recovery path for a failed image. The firmware must also handle interrupted transfers and reserve flash capacity for the update and recovery process.

Midnight Sun Vehicle Firmware

C / STM32 / CAN / WaveSculptor / FOTA

GitHub: [uw-midsun/fwxfv](#) Front controller FOTA

My work covers front-controller cruise-control integration and firmware updates for vehicle MCUs. I fixed safety state-machine faults that blocked WaveSculptor CAN setpoint tests and built a bootloader with CRC32 validation, sequencing, and ACK retries.



Steering wheel and vehicle controllers during bench integration.

From driver input to a permitted motor command

The steering wheel toggles cruise control when both cruise buttons are pressed, captures the current speed, and adjusts the setpoint through individual button presses. The front controller checks brake state, precharge completion, and battery-protection faults before allowing the request to become a motor command.

I focused on FSM faults blocking that command path. The WaveSculptor interface packs velocity and current commands as two 32-bit floats in an eight-byte CAN frame. Separating state decisions from serialization helps locate failures in the requested mode, drive state, or CAN output. Validation is currently at the bench-integration and setpoint-test stage.

A bootloader above reusable UART/CAN transport

Receive callbacks feed circular buffers; foreground code finds packet boundaries and reassembles accepted data for flash writes. Keeping parsing and flash operations outside receive callbacks limits interrupt work and lets the update state machine operate above either bus.

Update stage	Check or action
Framing and sequencing	Validate packet boundaries and expected chunk order
CRC32 and ACK retries	Detect corruption and recover missing transfers
Staging writes	Check bounds and accumulate the incoming image in flash
Whole-image verification	Check the staged image CRC before copying it into the application region

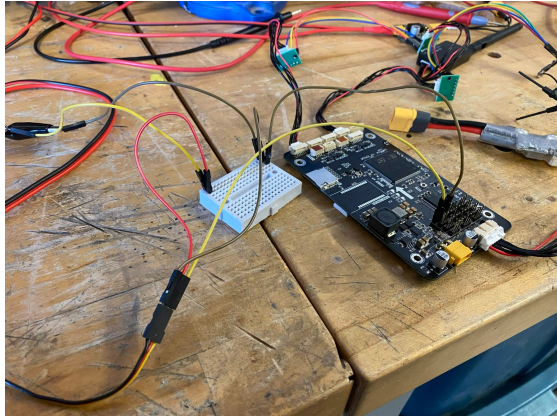
Staging permits a whole-image integrity check before application replacement, at the cost of flash capacity and a copy step. CRC32 detects accidental corruption; it does not authenticate the sender. Receive-buffer sizing must account for flash erase/write delays, and retries need bounded timeouts.

UWARG: ZeroPilot & LTE

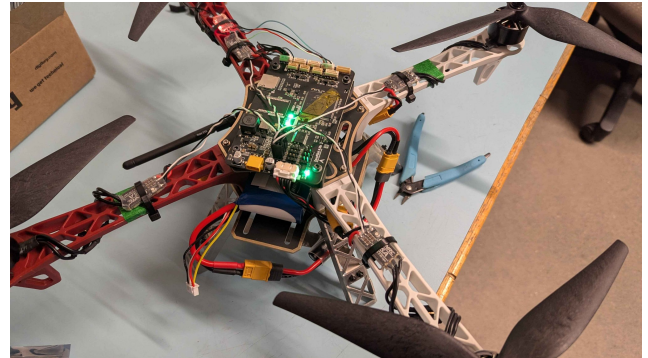
STM32H7 / C++ / FreeRTOS / PID / UART

GitHub: [UWARG/efs-zeropilot-4.0](https://github.com/UWARG/efs-zeropilot-4.0)

I implemented roll/yaw mixing and improved attitude-loop stability and actuator timing in ZeroPilot, UWARG's custom flight-control firmware. PID tuning reduced oscillation by 20% during stabilization tests; control-loop timing and output-scaling improvements reduced motor-command jitter by 30%.



ZeroPilot flight-controller hardware on the bench.



Quadcopter test platform. The mixing work below was for fixed-wing modes.

Roll-to-yaw mixing within actuator limits

In the fixed-wing FBWA path, pilot input becomes a bounded attitude setpoint. Roll and pitch controllers compare it with estimated attitude and scale their outputs into actuator commands. The roll command is centered around neutral before a configurable contribution is added to the rudder command, preventing a neutral offset from becoming an unintended rudder demand.

The final mixer maps commands to aileron, elevator, rudder, and throttle channels. Output clamping keeps commands within the actuator range; combined roll and yaw inputs near saturation limit how much mixing gain can be used without consuming the remaining rudder authority.

Fixing timing alongside gain tuning

The PID implementation uses its sample period in the integral and filtered derivative terms, with integral and output limits. I corrected control-loop timing and actuator scaling alongside gain tuning so the controller output matched the command sent to the actuator. This addressed sources of oscillation that gain changes alone could conceal.

A separate Quectel LTE driver

I also built a UART AT-command finite-state machine for the Quectel EG915Q modem and integrated it with a Raspberry Pi. Explicit command/response states make initialization progress visible. The parser must distinguish expected replies from unsolicited modem messages and handle responses that arrive fragmented or late.

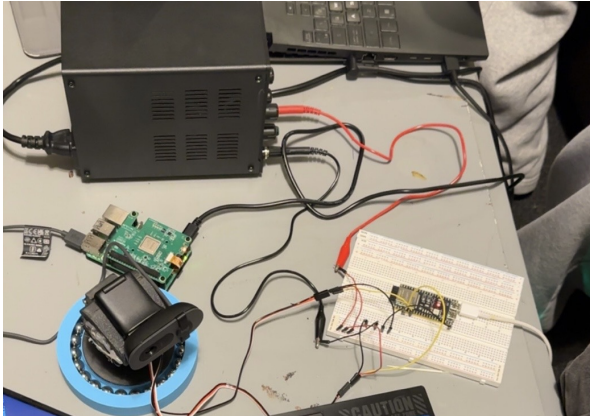
05 / HACKCANADA WINNER

remembR

Raspberry Pi 5 / Hailo-8L / YOLOv8s / Python / PCA9685

[GitHub: JJKSweaty/remembR](#) [Hailo RPi5 examples](#)

I built an edge AI memory-aid prototype that locates misplaced objects and returns their last-seen location and time to a phone interface. My work focused on the Pi/Hailo inference pipeline, persistent object memory, and pan-tilt camera hardware. The project won at HackCanada.



Pi, pan-tilt camera, and controller wiring on the bench.



Raspberry Pi and accelerator assembly.

Detection results become persistent object memory

The Raspberry Pi 5 and Hailo-8L accelerate YOLOv8s inference for a 1080p, 30 fps detection pipeline. A GStreamer callback extracts detection metadata and an optional frame, then pushes each batch into a bounded, thread-safe queue. A worker filters detections and updates timestamped object records; JSON persistence preserves recent history across restarts. FastAPI and WebSockets expose queries and snapshots over Tailscale.

When the queue fills, it discards the oldest batch to keep observations current. Empty batches also reach the worker so absence updates the temporal filter. A three-hits-in-five-frames rule suppresses flicker while adding a short confirmation delay.

Pan-tilt scanning with PCA9685 PWM

I drove dual-axis pan-tilt servos through a PCA9685 PWM controller to sweep the camera and recover objects outside its current view. Hardware PWM maintains the servo pulse train while the Pi processes camera frames. Camera motion expands coverage, with settling time affecting when a new observation becomes useful.

What an object query returns

A query returns current visibility, last-seen time, confidence, an image region, and an annotated snapshot. Records are grouped by class label, so two objects of the same class are not reliably distinguished. Location descriptions refer to the camera image rather than mapped room coordinates. Wallet searches use the model's handbag class as a proxy, which limits recognition accuracy for that query.

Hack The North: Badge Market

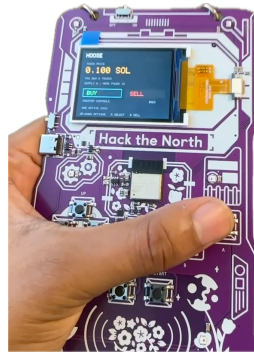
ESP32-C3 / C++17 / ESP-IDF / ESP-NOW

[GitHub: JJKSweaty/badge-market](#) [Verification notes](#) [Video demo](#)

I built a shared game where nearby players earn fictional currency, launch coins, and trade through their Hack the North badges. I moved it from a stock-runtime Lua app to native C++ firmware with board drivers, a display renderer, ESP-NOW transport, and persistent saves. One badge hosts the market without a phone or internet connection during play.



Market home screen.



Buy and sell controls.



MEMEWORD puzzle input.

State synchronization over ESP-NOW

Only the host commits trades. Guests retry pending requests with the same sequence number and payload, preventing a lost reply from applying a trade twice. Receive callbacks copy data into a 16-packet queue for later processing. Market snapshots use 200-byte chunks with revision, length, offset, and CRC checks; partial snapshots never become visible state. There is no host migration, so trading depends on the host remaining nearby.

DMA rendering within the RAM budget

A 320×240 RGB565 framebuffer would use 153,600 bytes. I used two 5 KiB DMA stripe buffers instead, hashing eight-row stripes and sending changed regions over SPI. This reduces RAM use and display traffic while requiring invalidation of moved or removed graphics.

Host and solo modes each keep two NVS snapshot slots. Saving writes the older slot, commits it, and reads it back. Checkpoints occur at gameplay-safe points; an unexpected power loss can roll back recent changes. Reward and transaction sequencing prevent duplicate credits during retries.

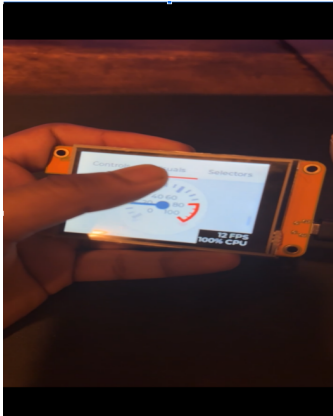
07 / PERSONAL PROJECT

ESP32 Media Controller

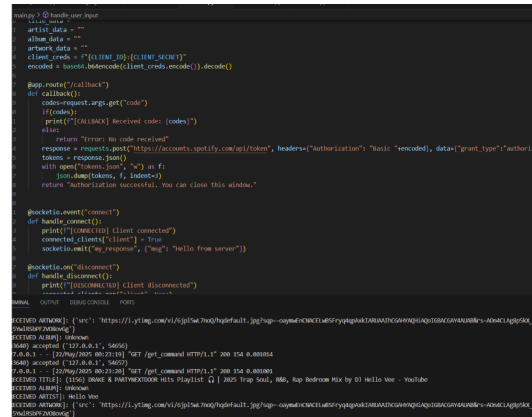
ESP32 / C++ / LVGL / FreeRTOS / Python / TCP

[GitHub: JJKSweaty/ESP32Media](#) [Video demo](#)

I built a handheld touchscreen dashboard for PC metrics, media state, Discord call information, and playback controls. A Python service collects desktop data; the ESP32 receives snapshots over Wi-Fi and sends commands through a TCP connection.



Media interface on the physical touchscreen.



Python backend supplying desktop updates.

Network work stays outside the UI path

The 320×240 display uses LVGL partial rendering and an XPT2046 touch controller. A lower-priority FreeRTOS task handles network parsing while the main loop services LVGL. A one-element overwrite queue retains only the newest pending snapshot, avoiding a backlog of stale CPU usage or track-position updates.

Newline-delimited JSON supplies message boundaries on the TCP byte stream. Touch actions enter a separate bounded command queue so the UI does not wait for socket writes. The receiver limits work per iteration and yields to keep network parsing from monopolizing execution.

Predictable memory for album artwork

Artwork arrives as base64-encoded 80×80 RGB565 pixels and decodes into one reusable 12,800-byte buffer. Keeping it outside the snapshot queue avoids copying image data with every update and provides a format LVGL can render directly. The encoded payload is 17,068 characters before JSON overhead, trading bandwidth for a simple decode and display path.

The network task skips artwork when heap is low and reconnects after inactivity. Outbound commands can be dropped when their queue fills; application-level acknowledgements would be needed to confirm delivery.

Panel and touch bring-up

I corrected display byte order and inversion settings for the panel and mapped raw touch readings into screen coordinates. Checking those interfaces separately helped distinguish rendering faults from display configuration or touch-calibration problems.